

Proyecto Grupal 01

Modelado de **Interconnect** para el análisis de desempeño de Protocolos de Coherencia HW/SW

Fecha de asignación: 26 marzo 2026
Grupos: 3 personas (máx)

Fecha de entrega: 05 mayo 2026
Profesores: Ronald García Fernández
Luis Chavarría Zamora

1. Objetivo

Mediante el desarrollo de este proyecto, las personas estudiantes aplicarán los conceptos de arquitectura de multiprocesadores (MP). El objetivo principal consiste en diseñar e implementar un modelo de simulación de un *interconnect* para analizar el impacto en el rendimiento de los protocolos de coherencia de caché frente a patrones de acceso a memoria de algoritmos paralelos.

2. Descripción general

Los sistemas computacionales modernos **comúnmente** se basan en múltiples núcleos que comparten memoria, los cuales se comunican mediante un *interconnect*. En los entornos de memoria compartida, un elemento fundamental para garantizar la consistencia de los datos es el protocolo de coherencia [2], el cual tiene un impacto sobre el tráfico a través del *interconnect*.

En este proyecto se implementará un conjunto de modelos que permitan integrar la perspectiva del software (*memory traces* generadas por algoritmos) con la perspectiva del hardware (el arbitraje y las transacciones en el bus). El equipo desarrollará un entorno de pruebas para comparar como decisiones algorítmicas impactan el consumo de ancho de banda en una arquitectura de hardware específica.

3. Especificación Técnica

La implementación del modelo debe seguir estrictamente las siguientes restricciones sobre la implementación para asegurar la validez experimental:

1. **Lenguajes Permitidos:** El diseño de hardware (*interconnect* y cachés) debe desarrollarse exclusivamente utilizando **SystemC** [5] (TLM) o **SystemVerilog** [6] (no sintetizable). Queda prohibido el uso de Python para el modelo de hardware.
2. **Arquitectura del Sistema:** El sistema base debe simular una topología que incluya un **mínimo de cuatro Elementos de Procesamiento (PEs)**, Cachés L1 privadas y un *Interconnect* y memoria compartida (ver Figura 1).

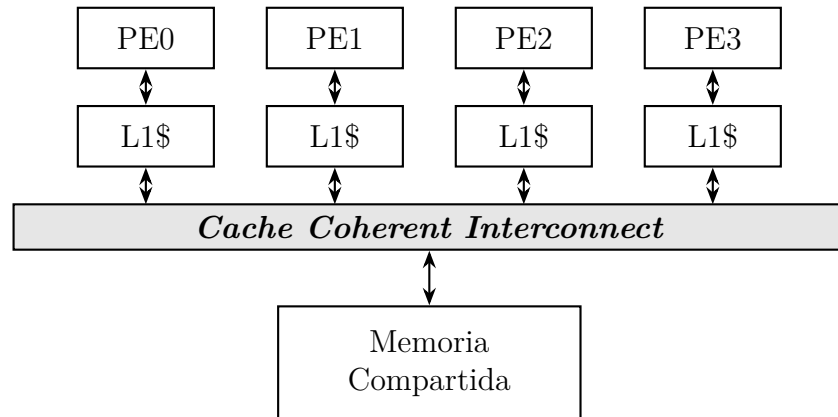


Figura 1: Ejemplo de topología del sistema multiprocesador con memoria compartida.

3. **Diseño del Elemento de Procesamiento (PE):** El PE operará como un *Trace Player* que lee comandos generados desde un script externo y los ejecuta según corresponda. No se requiere la implementación de un pipeline de instrucciones.
4. **Diseño del Interconnect:** El equipo debe definir y justificar la implementación *Interconnect* respecto a su esquema de arbitración, capacidad de manejo de tráfico, se recomienda modelar contadores de despempeño para medir diferentes eventos en el sistema.
5. **Arquitectura de Jerarquía de Memoria e Investigación:**
 - a) **Dimensionamiento Investigativo:** El equipo debe investigar y proponer la arquitectura de la jerarquía de memoria del sistema (caché de datos, memoria compartida) sobre su tamaño total, tamaño de bloque y asociatividad, latencia, etc. Esta configuración debe justificarse para asegurar que las *traces* generen suficientes reemplazos (*evictions*) y conflictos para validar la teoría.
 - b) **Protocolos:** Se debe investigar e implementar un protocolo de la familia *Write-Invalidate* (por ejemplo, el protocolo base MSI [3, 1]) y otro de la familia *Write-Broadcast/Update* (por ejemplo, el protocolo Firefly [4, 3]) para su respectiva comparación.
6. **Diseño de Transacciones:** El equipo debe definir y justificar un conjunto de transacciones soportadas por los PEs e *Interconnect*. Se permite la implementación de transacciones avanzadas siempre que su beneficio sea justificado con base en criterios de diseño.
7. **Cargas de Trabajo (Workloads):** Se deben modelar al menos dos escenarios paralelos basados en algoritmos o aplicaciones reales (ej. Alta Contención, Productor-Consumidor y Patrón Migratorio), esto con el fin de evaluar el desempeño y cambios en el tráfico del sistema.

8. **Mapeo y captura de transiciones de estado:** El sistema permitirá al usuario capturar las transiciones de estado del protocolo escogido para mostrarlas de forma intuitiva, según lo defina el usuario.

Los equipos de trabajo pueden usar herramientas de inteligencia artificial u otras fuentes como guía. Deben documentar los procedimientos automatizados y los mismos deben ser validados por usted contra referencias verificadas y confiables ¹. **En este proyecto usted será un diseñador(a), no un copiloto(a) sin criterio técnico crítico. Esto debe formar parte de los anexos en el documento entregado.**

4. Demostraciones de Avance

Se establecen las siguientes demostraciones de carácter **obligatorio** para analizar el progreso y eliminar bloqueos técnicos. Estas son obligatorias para la defensa final:

1. **Demostración 1 (jueves 9 de abril del 2026):** Conformación de equipos de trabajo, asignación de roles e investigación de posibles *workloads* basados en problemas, algoritmos o aplicaciones comunes en sistemas computacionales (mediante correo electrónico). En esta demostración, los estudiantes acordarán con el profesor acerca del manejo del repositorio GIT.
2. **Demostración 2 (martes 21 de abril del 2026):** Presentación funcional de los generadores de *memory traces* (software) y diseño arquitectónico de la caché L1 e investigación de transacciones (hardware). Revisión de ramas en Git.

5. Entregables

Como entregables en este proyecto se evaluará lo siguiente:

- **Presentación funcional final de 30 minutos (60 %):** El equipo deberá demostrar el funcionamiento del simulador integrado con un **mínimo de 4 PEs** operando en paralelo. Durante la sesión se evaluará:
 - Ejecución de las tres *memory traces* de software frente a ambos protocolos de coherencia.
 - Trazabilidad de transacciones en el bus mediante bitácoras o *waveforms*.
 - Despliegue de estadísticas de ancho de banda y rendimiento general.

¹Algunas veces estas herramientas inducen a sesgos y no deben ser tomadas sin cuestionamiento o verificación (como todo en ingeniería).

- **Artículo tipo *paper* en formato PDF (20 %):** Documento de máximo 4 páginas en formato IEEE. Debe incluir la justificación técnica de la arquitectura de caché propuesta y el análisis comparativo de resultados.
- **Video de presentación (20 %):** Video de entre 3 y 5 minutos. Debe exponer de forma intuitiva sus hallazgos, inspirándose en el estilo de divulgación del canal, y todas las personas del grupo deben participar. *Two Minute Papers* [7].
- **Opcional uso de inglés:** Se podrá optar por un 10 % extra sobre la nota final del proyecto únicamente si el artículo y el video se realizan en su totalidad en inglés y demuestran fluidez técnica.

Si tienen dudas pueden escribir a los profesores correspondientes:

- Luis Chavarría: lachavarria@tec.ac.cr
- Ronald García: ronald.cursos.tec@gmail.com

Los documentos serán sometidos a control de plagios. Trabajos con copias de semestres anteriores, código actual de otros grupos o copias textuales de repositorios de terceros tendrán nota de cero. La entrega se debe realizar por medio de la plataforma TEC-Digital en la sección de evaluaciones.

6. Referencias

Referencias

- [1] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Elsevier.
- [2] Nagarajan, V., et al. (2020). *A Primer on Memory Consistency and Cache Coherence*. Morgan & Claypool.
- [3] Archibald, J., & Baer, J.-L. (1986). *Cache Coherence Protocols: Evaluation Using a Multiprocessor Simulation Model*. ACM Transactions on Computer Systems (TOCS).
- [4] Thacker, C. P., Stewart, L. C., & Satterthwaite Jr, E. H. (1988). *Firefly: A Multiprocessor Workstation*. IEEE Transactions on Computers.
- [5] IEEE. (2024). *SystemC Language Reference Manual*. IEEE Std 1666-2023.
- [6] IEEE. (2018). *SystemVerilog Language Reference Manual*. IEEE Std 1800-2017.

- [7] Zsolnai-Fehér, K. *Two Minute Papers*. Canal oficial de YouTube:
<https://www.youtube.com/channel/UCbfYPyITQ-7l4upoX8nvctg>